DevDocs — Project & Conversation Decisions

1. Project Deletion

When a project is deleted, **everything associated with that project should also be deleted**.

The project acts as the ownership boundary for all of its data.

Cascade deletion

```
Project
   │
   ├── Files
   │     ├── File metadata
   │     └── Uploaded files
   │
   ├── Chunks
   │
   ├── Embeddings
   │
   └── Conversations
         └── Messages
```

Deleting the project should therefore remove:

- Project metadata
- File metadata
- Uploaded project files
- Ingestion information
- Chunks
- Embeddings/vector records
- Conversations
- Messages belonging to those conversations

There should not be orphaned data remaining after project deletion.

Core principle

> A project owns everything associated with it, so deleting the project should cascade through all of its dependent data.

---

2. Project ID Must Remain Unchanged When Renaming

A project has a stable internal identifier:

```
project_id = 101
name = "E-Commerce API"
```

If the user renames the project:

```
name = "My E-Commerce Backend"
```

the `project_id` remains:

```
project_id = 101
```

Why?

The project ID represents the **identity of the project**, while the name is only a human-readable attribute.

The name can change.

The identity should remain stable.

For example:

```
Before:

project_id = 101
name = "E-Commerce API"


After rename:

project_id = 101
name = "My E-Commerce Backend"
```

Only the name changes.

Why changing the ID would be unnecessary

Many pieces of data can reference the project:

```
Files
   ↓
project_id

Chunks
   ↓
```

```
project_id

Embeddings
   ↓
project_id

Conversations
   ↓
project_id
```

If the project ID changed during a rename, all of those relationships would unnecessarily need to be updated.

Keeping the ID stable avoids this.

Core principle

> **Project name is mutable. Project ID is stable.**

This also gives us a clean foundation for future functionality such as GitHub synchronization, project versions, background processing, and other project-level features.

---

3. Multiple Conversations Per Project

A project should support **multiple independent conversations**.

We should not have one giant chat history for the entire project.

Instead:

```
Project
   │
   ├── Conversation 1
   │       └── Messages
   │
   ├── Conversation 2
   │       └── Messages
   │
   └── Conversation 3
           └── Messages
```

Example

Project:

```
E-Commerce API
```

Could contain:

```
Conversations

• How does authentication work?
• Explain the payment flow
• Why is order creation failing?
• Explain the database architecture
• How does JWT validation work?
```

All conversations belong to the same project and therefore have access to the same project knowledge through project-scoped RAG.

However, each conversation maintains its own message history.

Important distinction

The project determines:

> **What knowledge DevDocs can retrieve.**

The conversation determines:

> **What discussion the user is currently having.**

---

4. Restoring Previous Conversations

When a user opens an existing conversation, DevDocs should restore the previous conversation history.

For example:

```
Conversation
──────────────────────────

User:
How does authentication work?

DevDocs:
Authentication is handled by...

User:
Where is the JWT generated?
```

```
DevDocs:
The JWT is generated in...

User:
What happens after that?
```

The user should be able to continue the conversation from where they left off.

Important distinction

Restoring a conversation in the UI does **not** mean sending the entire conversation history to the LLM.

These are two different operations.

UI restoration

The application can retrieve the stored messages:

```
Conversation
     ↓
Database
     ↓
Messages
     ↓
UI
```

This is a normal database operation.

It does not require:

- LLM calls
- Embedding generation
- Vector search
- RAG processing

Therefore, restoring an old conversation should be relatively inexpensive.

---

5. Conversation Context Must Have a Limit

A conversation should **not** continuously grow into a massive LLM context.

For example, we should avoid:

```
100 messages
     ↓
Entire conversation
```

```
     ↓
Huge prompt
     ↓
LLM
```

This can cause:

- Larger prompts
- Higher token usage
- Higher latency
- Higher cost
- More irrelevant information
- Context-window pressure
- Potentially worse answers

Instead, DevDocs should maintain a controlled amount of conversational context.

---

6. Proposed Conversation Context Strategy

The conceptual approach is:

```
Older conversation
       ↓
Conversation summary
       │
       ├── Important decisions
       ├── Important conclusions
       ├── Relevant user intent
       └── Important context

Recent messages
       ↓
Current question
       ↓
Project-scoped RAG
       ↓
LLM
```

Therefore, the LLM does **not** need the complete historical conversation.

It receives:

1. A summary of important older context
2. A limited number of recent messages
3. Relevant project knowledge retrieved through RAG
4. The user's current question

---

7. Complete Conversation vs LLM Context

The application can still preserve the **complete conversation**.

For example:

```
Database

Conversation
 ├── Message 1
 ├── Message 2
 ├── Message 3
 ├── ...
 ├── Message 97
 ├── Message 98
 ├── Message 99
 └── Message 100
```

The UI can display all of these messages when the user opens the conversation.

But when the user sends message 101, we do not necessarily send all 100 previous messages to the LLM.

Instead:

```
Stored conversation
       │
       ├── Older messages
       │       ↓
       │    Summary
       │
       └── Recent messages
               ↓

Project RAG
       ↓
Relevant project chunks
       ↓
Context builder
       ↓
LLM
```

This separates **conversation storage** from **LLM context management**.

---

8. Example

Imagine a developer has been discussing authentication for a long time.

The conversation contains 80 messages.

The application could retain all 80 messages in the database.

However, the LLM might receive something closer to:

```
Conversation summary:

The project uses JWT authentication.
AuthService generates the JWT after validating credentials.
JWT validation occurs through AuthMiddleware.
The developer is currently investigating duplicate validation.

Recent messages:

User: Why is the token being validated twice?

DevDocs: ...

User: Which files are responsible for the second validation?

Current question:

Where exactly should I modify the implementation?
```

Then RAG retrieves relevant project chunks such as:

```
backend/auth.py
backend/middleware/auth.py
backend/services/auth_service.py
```

The LLM receives only the information relevant to answering the current question.

---

9. Cost Consideration

Restoring an old conversation is **not inherently an expensive GenAI operation**.

Loading stored messages from a database is simply a database read.

For example:

```
Open conversation
      ↓
Fetch messages
      ↓
Display messages
```

No LLM call is required.

The expensive operations are primarily associated with generating a new answer, such as:

```
User question
      ↓
RAG retrieval
      ↓
Prompt construction
      ↓
LLM call
      ↓
Answer
```

Therefore:

> **Showing conversation history is cheap. Sending excessive conversation history to the LLM is what we want to avoid.**

---

10. Important Architecture Distinction

DevDocs will have two different concepts:

Conversation storage

Responsible for preserving the user's complete conversation.

```
Database
    ↓
Complete conversation history
```

Conversation context

Responsible for deciding what portion of that history is actually provided to the LLM.

```
Complete history
       ↓
Context management
```

```
       ↓
Summary + recent messages
       ↓
LLM
```

This allows DevDocs to preserve the user's history without creating an ever-growing LLM context.

---

11. Decisions Locked So Far

The following decisions are now finalized:

Project

- Project deletion will cascade.
- Deleting a project removes all associated data.
- `project_id` remains unchanged when the project is renamed.
- Project ID is the stable identity of a project.

Conversations

- A project can have multiple conversations.
- Each conversation belongs to exactly one project.
- Each conversation has its own message history.
- Users can reopen and continue previous conversations.
- Complete conversation history is stored.
- Conversation history should not be sent entirely to the LLM.

Conversation Context

- LLM context will have a practical limit.
- Recent messages will be prioritized.
- Older conversation information can be summarized.
- Project knowledge should continue to come from project-scoped RAG.
- Restoring conversation history itself should not require an LLM call.

---

12. Next Design Area

Before implementing the RAG/chat system, we should design the **conversation system** in more detail.

The next questions to settle are:

1. What exactly constitutes a conversation?
2. What data should a conversation store?
3. How should individual messages be stored?
4. How should conversation history interact with RAG?
5. When should older messages be summarized?
6. What happens when the user starts a new conversation?

7. What happens when a conversation is deleted?
8. How should conversations appear in the DevDocs UI?
9. Most importantly: **Should conversational memory contain project knowledge, or should project knowledge always come from RAG?**

The last question is particularly important because it determines the boundary between:

```
Conversation Memory
        +
Project Knowledge
        ↓
       LLM
```

We should decide this before implementing the actual RAG/chat architecture.